# 📖 FinEdge API Documentation

Complete API reference for the FinEdge Personal Finance Management Application.

**Base URL:** `http://localhost:3000`

---

## 📑 Table of Contents



---

## 🔐 Authentication

FinEdge uses **JWT (JSON Web Tokens)** for authentication. After registration or login, you'll receive a token that must be included in subsequent requests.

### Bearer Token Authentication

Include the JWT token in the `Authorization` header:

```
Authorization: Bearer <your_jwt_token>
```

**Token Expiration:** 7 days (configurable via `JWT_EXPIRES_IN` env variable)

---

## 👤 User Endpoints

### 1. Register New User

Creates a new user account and returns a JWT token.

**Endpoint:** `POST /users`

**Authentication:** Not required

**Request Body:**

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "password123"
}
```

**Validation Rules:**

- `name` : 3-30 characters, alphanumeric only
- `email` : Valid email format

- `password` : 8-30 characters

**Success Response:** `201 Created`

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "user": {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d1",
      "name": "John Doe",
      "email": "john@example.com",
      "createdAt": "2026-03-09T10:30:00.000Z",
      "updatedAt": "2026-03-09T10:30:00.000Z"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

**Error Responses:**

`400 Bad Request` - Validation error

```
{
  "success": false,
  "error": "Name must be at least 3 characters long"
}
```

`400 Bad Request` - User already exists

```
{
  "success": false,
  "error": "User already exists"
}
```

---

### 2. Login User

Authenticates a user and returns a JWT token.

**Endpoint:** `POST /users/login`

**Authentication:** Not required

**Request Body:**

```
{
  "email": "john@example.com",
  "password": "password123"
}
```

**Success Response:** `200 OK`

```
{
  "success": true,
  "message": "User logged in successfully",
  "data": {
    "user": {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d1",
      "name": "John Doe",
      "email": "john@example.com"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```


**Error Responses:**

`401 Unauthorized` - Invalid credentials


```
{
  "success": false,
  "error": "Invalid credentials"
}
```


`400 Bad Request` - Validation error


```
{
  "success": false,
  "error": "Please enter a valid email address"
}
```


---

### 3. Get User by ID

Retrieves a specific user's information.

**Endpoint:** `GET /users/:id`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `id` (string, required): User ID

**Success Response:** `200 OK`


```
{
  "success": true,
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d1",
    "name": "John Doe",
    "email": "john@example.com",
    "createdAt": "2026-03-09T10:30:00.000Z",
    "updatedAt": "2026-03-09T10:30:00.000Z"
```

```
  }
}
```

**Error Responses:**

`401 Unauthorized` - Missing or invalid token

```
{
  "success": false,
  "error": "Unauthorized"
}
```

`404 Not Found` - User not found

```
{
  "success": false,
  "error": "User not found"
}
```

---

### 4. Get All Users

Retrieves a list of all users.

**Endpoint:** `GET /users`

**Authentication:** Required (Bearer Token)

**Success Response:** `200 OK`

```
{
  "success": true,
  "message": "All users found",
  "data": [
    {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d1",
      "name": "John Doe",
      "email": "john@example.com",
      "createdAt": "2026-03-09T10:30:00.000Z"
    },
    {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d2",
      "name": "Jane Smith",
      "email": "jane@example.com",
      "createdAt": "2026-03-08T15:20:00.000Z"
    }
  ]
}
```

---

## 💰 Transaction Endpoints

### 5. Create Transaction

Creates a new income or expense transaction.

**Endpoint:** `POST /api/v1/transactions`

**Authentication:** Required (Bearer Token)

**Request Body:**

```
{
  "type": "expense",
  "category": "Groceries",
  "amount": 150.50,
  "date": "2026-03-09"
}
```

**Fields:**

- `type` (string, required): `income` or `expense`
- `category` (string, required): Transaction category
- `amount` (number, required): Positive number
- `date` (string, optional): ISO date format (defaults to current date)

**Auto-Categorization:**

The system automatically suggests categories based on keywords:

- `Groceries` : walmart, grocery, supermarket, food, market
- `Dining` : restaurant, cafe, pizza, mcdonald, starbucks
- `Transportation` : uber, lyft, gas, fuel, parking
- `Entertainment` : movie, cinema, netflix, spotify
- `Utilities` : electric, water, internet, phone
- `Shopping` : amazon, clothing, mall

**Success Response:** `201 Created`

```
{
  "success": true,
  "message": "Transaction created successfully",
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d3",
    "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
    "type": "expense",
    "category": "Groceries",
    "amount": 150.5,
    "date": "2026-03-09T00:00:00.000Z",
    "createdAt": "2026-03-09T10:35:00.000Z",
    "updatedAt": "2026-03-09T10:35:00.000Z"
  }
}
```

**Auto-Corrected Response:** `201 Created`

```
{
  "success": true,
  "message": "Transaction created. Category auto-corrected to 'Groceries'.",
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d3",
    "type": "expense",
    "category": "Groceries",
    "amount": 45.99
  }
}
```


**Error Responses:**

`400 Bad Request` - Validation error


```
{
  "success": false,
  "error": "Amount must be a positive number"
}
```


`401 Unauthorized` - Missing token


```
{
  "success": false,
  "error": "Unauthorized"
}
```


---

### 6. Get All Transactions

Retrieves all transactions for the authenticated user.

**Endpoint:** `GET /api/v1/transactions`

**Authentication:** Required (Bearer Token)

**Success Response:** `200 OK`


```
{
  "success": true,
  "message": "Transactions retrieved successfully",
  "data": [
    {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d3",
      "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
      "type": "income",
      "category": "Salary",
      "amount": 5000,
      "date": "2026-03-01T00:00:00.000Z"
    },
    {
```

```
      "_id": "65f7a8b9c4d5e6f7a8b9c0d4",
      "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
      "type": "expense",
      "category": "Groceries",
      "amount": 150.5,
      "date": "2026-03-09T00:00:00.000Z"
    }
  ]
}
```

---

### 7. Get Transaction by ID

Retrieves a specific transaction.

**Endpoint:** `GET /api/v1/transactions/:transactionId`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `transactionId` (string, required): Transaction ID

**Success Response:** `200 OK`

```
{
  "success": true,
  "message": "Transaction retrieved successfully",
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d3",
    "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
    "type": "expense",
    "category": "Groceries",
    "amount": 150.5,
    "date": "2026-03-09T00:00:00.000Z"
  }
}
```

**Error Responses:**

`404 Not Found` - Transaction not found

```
{
  "success": false,
  "error": "Transaction not found"
}
```

---

### 8. Update Transaction

Updates an existing transaction.

**Endpoint:** `PATCH /api/v1/transactions/:transactionId`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `transactionId` (string, required): Transaction ID

**Request Body:** (all fields optional)

```
{
  "type": "expense",
  "category": "Food",
  "amount": 175.00,
  "date": "2026-03-09"
}
```

**Success Response:** `200 OK`

```
{
  "success": true,
  "message": "Transaction updated successfully",
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d3",
    "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
    "type": "expense",
    "category": "Food",
    "amount": 175,
    "date": "2026-03-09T00:00:00.000Z",
    "updatedAt": "2026-03-09T11:00:00.000Z"
  }
}
```

**Error Responses:**

`404 Not Found` - Transaction not found

```
{
  "success": false,
  "error": "Transaction not found"
}
```

`400 Bad Request` - Validation error

```
{
  "success": false,
  "error": "Amount must be a positive number"
}
```

---

### 9. Delete Transaction

Deletes a transaction.

**Endpoint:** `DELETE /api/v1/transactions/:transactionId`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `transactionId` (string, required): Transaction ID

**Success Response:** `204 No Content`

**Error Responses:**

`404 Not Found` - Transaction not found

```
{
  "success": false,
  "error": "Transaction not found"
}
```

---

## 💼 Budget Endpoints

### 10. Create Budget

Creates a new budget for the user.

**Endpoint:** `POST /api/v1/budgets`

**Authentication:** Required (Bearer Token)

**Request Body:**

```
{
  "monthlyGoal": 3000,
  "savingsTarget": 1000
}
```

**Fields:**

- `monthlyGoal` (number, required): Monthly spending limit
- `savingsTarget` (number, required): Savings goal

**Success Response:** `201 Created`

```
{
  "success": true,
  "message": "Budget created successfully",
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d5",
    "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
    "monthlyGoal": 3000,
    "savingsTarget": 1000,
    "createdAt": "2026-03-09T10:40:00.000Z"
```

```
  }
}
```

---

### 11. Get All Budgets

Retrieves all budgets for the authenticated user.

**Endpoint:** `GET /api/v1/budgets`

**Authentication:** Required (Bearer Token)

**Success Response:** `200 OK`

```
{
  "success": true,
  "data": [
    {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d5",
      "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
      "monthlyGoal": 3000,
      "savingsTarget": 1000,
      "createdAt": "2026-03-09T10:40:00.000Z"
    }
  ]
}
```

---

### 12. Get Budget by ID

Retrieves a specific budget.

**Endpoint:** `GET /api/v1/budgets/:budgetId`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `budgetId` (string, required): Budget ID

**Success Response:** `200 OK`

```
{
  "success": true,
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d5",
    "userId": "65f7a8b9c4d5e6f7a8b9c0d1",
    "monthlyGoal": 3000,
    "savingsTarget": 1000,
    "createdAt": "2026-03-09T10:40:00.000Z"
  }
}
```

---

### 13. Update Budget

Updates an existing budget.

**Endpoint:** `PATCH /api/v1/budgets/:budgetId`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `budgetId` (string, required): Budget ID

**Request Body:** (all fields optional)

```
{
  "monthlyGoal": 3500,
  "savingsTarget": 1200
}
```

**Success Response:** `200 OK`

```
{
  "success": true,
  "message": "Budget updated successfully",
  "data": {
    "_id": "65f7a8b9c4d5e6f7a8b9c0d5",
    "monthlyGoal": 3500,
    "savingsTarget": 1200,
    "updatedAt": "2026-03-09T11:10:00.000Z"
  }
}
```

---

### 14. Delete Budget

Deletes a budget.

**Endpoint:** `DELETE /api/v1/budgets/:budgetId`

**Authentication:** Required (Bearer Token)

**URL Parameters:**

- `budgetId` (string, required): Budget ID

**Success Response:** `204 No Content`

---

## 📊 Analytics Endpoints

### 15. Get Financial Summary

Retrieves total income and expenses for the authenticated user.

**Endpoint:** `GET /api/v1/transactions/summary`

**Authentication:** Required (Bearer Token)

**Success Response:** `200 OK`

```
{
  "success": true,
  "message": "Transaction summary retrieved successfully",
  "data": {
    "totalIncome": 5000,
    "totalExpense": 1250.75
  }
}
```

**Note:** Summary data is cached for 60 seconds for performance optimization.

**Calculated Fields:**

- `totalIncome` : Sum of all income transactions
- `totalExpense` : Sum of all expense transactions
- Net Balance: `totalIncome - totalExpense` (calculate on client side)

---

### 16. Get Spending Recommendations

Get AI-powered spending recommendations based on transaction history.

**Endpoint:** `GET /api/v1/transactions/recommendations`

**Authentication:** Required (Bearer Token)

**Success Response:** `200 OK`

**Example 1: With recommendations**

```
{
  "success": true,
  "data": [
    {
      "category": "Dining",
      "currentSpending": 450,
      "lastMonthSpending": 300,
      "message": "Your Dining spending increased by 50.0%. Consider reducing to
$300.00."
    },
    {
      "category": "Transportation",
      "currentSpending": 250,
      "lastMonthSpending": 150,
      "message": "Your Transportation spending increased by 66.7%. Consider reducing
to $150.00."
    }
  ]
}
```

**Example 2: No issues found**

```
{
  "success": true,
  "data": [
    {
      "message": "Great job! Your spending is under control."
    }
  ]
}
```

**Example 3: No income recorded**

```
{
  "success": true,
  "data": [
    {
      "category": "Income",
      "message": "No income recorded this month. Add your income to track savings
better."
    }
  ]
}
```

**Recommendation Logic:**

- Compares current month spending with previous month
- Alerts if spending increased by more than 15% in any category
- Suggests reducing to previous month's spending level
- Notifies if no income recorded this month

---

### 17. Get Recent Transactions

Retrieves the 10 most recent transactions, optionally filtered by date.

**Endpoint:** `GET /api/v1/transactions/recent`

**Authentication:** Required (Bearer Token)

**Query Parameters:**

- `since` (string, optional): ISO timestamp to filter transactions after this date

**Example Request:**

```
GET /api/v1/transactions/recent?since=2026-03-01T00:00:00.000Z
```

**Success Response:** `200 OK`

```
{
  "success": true,
  "data": [
```

```
    {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d8",
      "type": "expense",
      "category": "Groceries",
      "amount": 85.50,
      "date": "2026-03-09T08:30:00.000Z"
    },
    {
      "_id": "65f7a8b9c4d5e6f7a8b9c0d7",
      "type": "income",
      "category": "Freelance",
      "amount": 500,
      "date": "2026-03-08T14:20:00.000Z"
    }
  ]
}
```

---

## ⚠️ Error Handling

All API endpoints follow a consistent error response format.

### Error Response Structure

```
{
  "success": false,
  "error": "Error message describing what went wrong",
  "statusCode": 400
}
```

### Common HTTP Status Codes

| Code | Meaning | Description |
|---|---|---|
| 200 | OK | Request successful |
| 201 | Created | Resource created successfully |
| 204 | No Content | Resource deleted successfully |
| 400 | Bad Request | Invalid request data or validation error |
| 401 | Unauthorized | Missing or invalid authentication token |
| 404 | Not Found | Resource not found |
| 429 | Too Many Requests | Rate limit exceeded |
| 500 | Internal Server Error | Server error occurred |

### Common Error Scenarios

#### 1. Validation Errors (400)

```
{
  "success": false,
  "error": "Name must be at least 3 characters long"
}
```

**2. Authentication Errors (401)**

```
{
  "success": false,
  "error": "Unauthorized"
}
```

**3. Resource Not Found (404)**

```
{
  "success": false,
  "error": "Transaction not found"
}
```

**4. Rate Limit Exceeded (429)**

```
{
  "error": "Too many requests, please try again later."
}
```

---

## 🔒 Rate Limiting

The API implements rate limiting to prevent abuse.

**Limits:**

- **100 requests per 15 minutes** per IP address
- Applies to all endpoints

**Rate Limit Headers:**

Response includes rate limit information:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1678350000
```

**When Exceeded:**

Status: `429 Too Many Requests`

```
{
  "error": "Too many requests, please try again later."
}
```

**Best Practices:**

- Implement exponential backoff on rate limit errors
- Cache responses when possible
- Use webhooks instead of polling where available

---

## 🧪 Testing the API with Postman

1. Import the API endpoints into Postman
2. Set up environment variables:
   - `baseUrl` : `http://localhost:3000`
   - `authToken` : Your JWT token (set after login)
3. Use `{{baseUrl}}` and `{{authToken}}` in requests

**Auto-save token after login:**

Add to Login request's **Tests** tab:

```
var jsonData = pm.response.json();
pm.environment.set("authToken", jsonData.data.token);
```

---

## 📝 API Best Practices

1. **Always use HTTPS in production**
2. **Store JWT tokens securely** (not in localStorage for sensitive apps)
3. **Refresh tokens before expiration** (tokens expire after 7 days)
4. **Handle errors gracefully** with proper error messages
5. **Implement request timeouts** on client side
6. **Validate data on client side** before sending requests
7. **Use appropriate HTTP methods** (GET, POST, PATCH, DELETE)
8. **Monitor rate limits** and implement backoff strategies

---

## 🔗 Quick Reference

### Base URLs

- **Development:** `http://localhost:3000`
- **Production:** `https://your-domain.com`

### Authentication

- **Header:** `Authorization: Bearer <token>`
- **Token Expiry:** 7 days

### Response Format

- **Success:** `{ "success": true, "data": {...} }`
- **Error:** `{ "success": false, "error": "message" }`

### Rate Limits

- **Limit:** 100 requests / 15 minutes
- **Scope:** Per IP address

---

**Last Updated:** March 9, 2026
**API Version:** 1.0.0